

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

```
# =====
```

```
# NAIVE BAYES CLASSIFIER
```

```
# =====
```

```
# Import libraries
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

```
from sklearn.datasets import load_iris
```

```
# Load dataset
```

```
data = load_iris()
```

```
# Features
```

```
X = data.data
# Target
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.3, random_state=42 )

# Create Naive Bayes model
model = GaussianNB()

# Train model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Calculate precision
precision = precision_score(
    y_test,
    y_pred,
    average="weighted"
)
```

```

# Calculate recall
recall = recall_score(
    y_test,
    y_pred,
    average="weighted"
)

# Display results
print("Naive Bayes Classifier")
print("-----")

print("Accuracy :", accuracy)

print("Precision :", precision)

print("Recall :", recall)

```

OUTPUT:

Naive Bayes Classifier

Accuracy : 0.9777777777777777

Precision : 0.9793650793650793

Recall : 0.9777777777777777

2. Perform an experiment to estimate parameters of a probability

distribution using

Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation

approach. Use a dataset to compute parameters such as mean and variance, and

observe how prior assumptions influence results. Analyze differences in estimation

under small and large sample sizes, and interpret the impact on model reliability.

```
# =====
```

```
# MLE AND BAYESIAN ESTIMATION
```

```
# =====
```

```
# Import libraries
```

```
import numpy as np
```

```
# Generate dataset
```

```
data = np.array([10, 12, 13, 15, 16, 18, 20, 22])
```

```
# -----
```

```
# MLE ESTIMATION
```

```
# -----
```

```
# Calculate MLE mean
```

```
mle_mean = np.mean(data)
```

```
# Calculate MLE variance
```

```
mle_variance = np.var(data)

print("MLE ESTIMATION")
print("-----")

print("MLE Mean :", mle_mean)

print("MLE Variance :", mle_variance)


# -----
# BAYESIAN ESTIMATION #
# -----

# Prior mean
prior_mean = 15

# Prior strength
prior_weight = 2

# Number of observations
n = len(data)

# Sample mean
sample_mean = np.mean(data)
# Bayesian mean
```

```
bayesian_mean = (  
    prior_weight * prior_mean + n * sample_mean  
) / (prior_weight + n)
```

```
# Display Bayesian result
```

```
print("\nBAYESIAN ESTIMATION")
```

```
print("-----")
```

```
print("Prior Mean :", prior_mean)
```

```
print("Sample Mean :", sample_mean)
```

```
print("Bayesian Mean :", bayesian_mean)
```

```
# -----
```

```
# COMPARISON
```

```
# -----
```

```
print("\nCOMPARISON")
```

```
print("-----")
```

```
print("MLE Mean :", mle_mean)
```

```
print("Bayesian Mean :", bayesian_mean)
```

```
print("MLE Variance :", mle_variance)
```

OUTPUT

MLE ESTIMATION

MLE Mean : 15.75

MLE Variance : 14.6875

BAYESIAN ESTIMATION

Prior Mean : 15

Sample Mean : 15.75

Bayesian Mean : 15.6

COMPARISON

MLE Mean : 15.75

Bayesian Mean : 15.6

MLE Variance : 14.6875